

binhttp: wire format

Binary request/response framing for bserve and bcurl. All lengths count bytes; every multi-byte integer is unsigned and big-endian. This is not HTTP text framing.

1. Transport and connection model

Use raw TCP, without TLS. A client opens **one connection per invocation**, with no retries, redirects or parallel connections. Exchanges strictly alternate: client REQUEST, server exactly one RESPONSE, then repeat on the same connection. No pipelining or multiplexing.

Keep-alive is the default. Either peer may close between exchanges; TCP FIN is the only termination signal. A close mid-frame is an abort. The client reports an error and exits non-zero if the peer closes while a response is still expected. There is no close frame or Connection: close behavior in v1.

2. Fixed frame header: exactly 12 bytes

Offset	Bytes	Field	Encoding / meaning
0	4	magic	ASCII BSC1: 42 53 43 31 (hex)
4	1	version	0x01
5	1	type	0x01 REQUEST; 0x02 RESPONSE
6	1	flags	Reserved and unused; send 0x00 in v1
7	1	reserved	Send 0x00; receivers ignore this byte
8	4	payload_length	uint32; payload bytes only, excluding the header

Read exactly 12 bytes. Validate **magic**, then **version**, then **payload_length**. Reject lengths above **16,777,216 bytes (16 MiB)** before reading or allocating the payload. Read exactly the declared payload length only after these checks pass. TCP reads may be partial; accumulate bytes until each field/frame is complete.

3. REQUEST payload: type 0x01

Offset	Bytes	Field / rule
0	1	method: 0x01 = GET; other methods receive 405
1	2	path_length: uint16, maximum 4096
3	path_length	path: UTF-8; must begin with /; root-relative
3 + path_length	Variable	headers: shared header list (section 6)

4. RESPONSE payload: type 0x02

Fields in order: **status** (uint16), **headers** (section 6), **body_length** (uint32), then **body** (exactly body_length bytes). The body is the file bytes, unchanged; non-2xx responses have an empty body. If the encoded header list occupies h bytes, body_length starts at offset 2 + h and the body starts at 6 + h.

5. Filesystem boundary

Resolve paths only under the configured server root. Normalize . and .. before filesystem access without allowing traversal above root; resolve symlinks and re-check containment. Return 404 unless the resolved path is a real file inside root. Do not distinguish blocked traversal from an ordinary missing file.

binhttp: parsing rules

Header compression, forward-compatible frame handling and error recovery. These rules apply to both peers unless server/client behavior is stated explicitly.

6. Shared header list encoding

Start with **header_count** (1 byte, maximum 64), followed by that many entries. Each entry begins with **name_id** (1 byte). IDs 0x00-0x09 select the static name below. ID 0xFF is a literal: append **name_len** (1 byte, maximum 255) and that many UTF-8 name bytes. IDs 0x0A-0xFE are invalid.

Every entry then has **value_len** (uint16, maximum 65,535), followed by that many UTF-8 value bytes. Length prefixes count encoded bytes, not characters. There is no dynamic table or Huffman coding.

ID	Static name	ID	Static name
0x00	Host	0x05	Server
0x01	Content-Type	0x06	User-Agent
0x02	Content-Length	0x07	Accept
0x03	Connection	0x08	Cache-Control
0x04	Date	0x09	Last-Modified

7. Frame types and the mandatory skip rule

0x01 is REQUEST (client); **0x02** is RESPONSE (server). **0x00** is invalid. Values **0x03-0x7F** are reserved for future core types; **0x80-0xFF** are reserved for extensions.

For an unrecognized frame type, **validate the header, read exactly payload_length bytes, discard them, and continue with the next frame**. Do not interpret the payload or return 400. Frame size comes from the fixed header, never the type. New types can be skipped under wire version 0x01; a different version still fails validation.

8. Status codes and error recovery

Status	Meaning / condition
200 OK	File found; body contains its exact bytes
400 Bad Request	Malformed frame or payload
404 Not Found	Missing/non-file path or a path outside the root boundary
405 Method Not Allowed	Method is not GET (0x01)
413 Payload Too Large	File exceeds the server's servable-size ceiling
500 Internal Server Error	Unexpected server-side failure

Bad magic, version or oversized frame length: server sends a best-effort 400, then closes; client reports an error and exits non-zero. **Malformed payload after a valid header:** consume the declared payload, send 400 and keep the connection open on the server; client reports an error and exits non-zero. **Mid-frame close:** abandon the incomplete frame and connection.

9. Design choices

The 12-byte header is three 32-bit words, easy to inspect in a hex dump, without a stream ID. A uint32 payload length fixes the frame boundary; the enforced 16 MiB limit, not the field width, bounds reads. Ignored reserved bytes and skippable unknown types leave room for extension.

Source: docs/SUBMISSION_SPEC.md; supporting detail: PROTOCOL.md. Repository: anushkabhansalii/network-architecture-assignment.